

Currently we have multiple methods for cryptography which have different purposes. The primary method would be Elliptic Curves for public-key cryptography, ensuring that information sent over the communication channel is well protected. The start and end device will use Hashing like SHA-512 and Hashed Message Authentication Codes (HMACs) to ensure integrity verification of the data.

Fig. 1: Authentication Channel

Authentication goal:

- Bob wants to be sure that only Alice could have sent the message

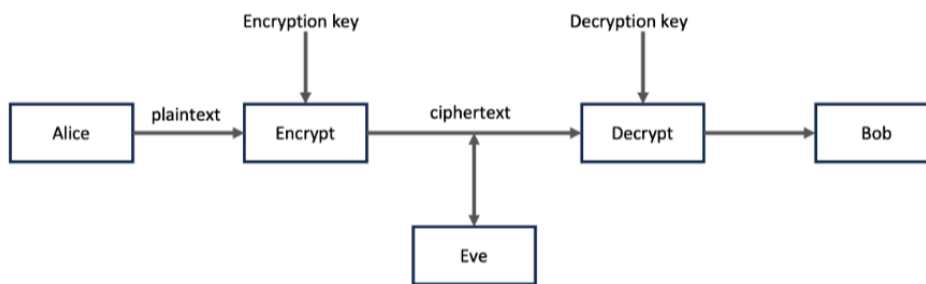
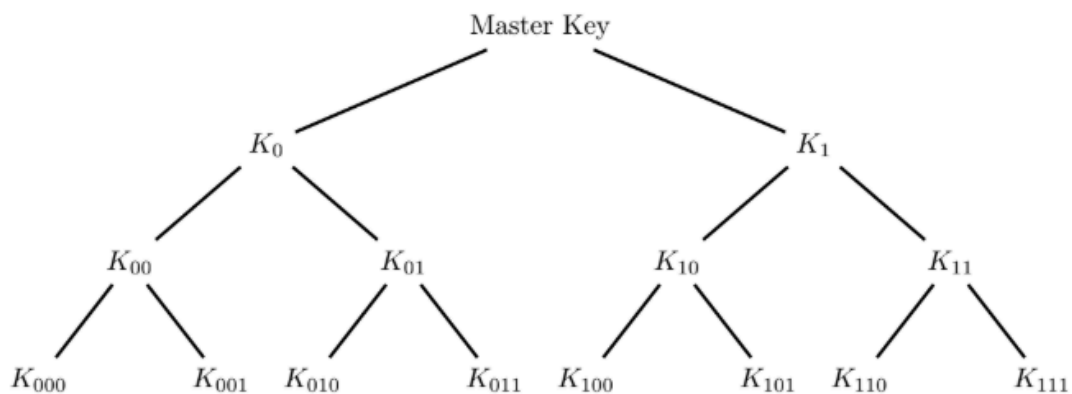


Fig. 2: Key Sharing Tree



Generated cryptographic secrets must be securely stored in nonvolatile storage to ensure data-at-rest protection. A PRF-based Key Derivation Tree (GGM Tree) will be used with global.secrets as the root node. Each subsequent child node is deterministically created through the extract-expand paradigm of HKDF. The group id is used by the HSM to derive the path of the Group Key. The user provides a 6-character hex PIN for each HSM called HSM_PIN. The verification value digest is calculated through HMAC(GroupKey, HSM_PIN) and it is stored in flash as a "golden reference" and recalculated from the tree whenever access is requested.

MPU:

A memory protection unit (MPU) can be utilized to enhance security, providing robust memory protection mechanisms. The MPU supports up to eight configurable memory regions, enabling granular access control. Additionally, the Sub Region Disable (SRD) feature allows for efficient use of memory regions by selectively disabling specific sub-regions. Furthermore, the MPU can enable a background region that enforces default memory map attributes, ensuring a secure and consistent memory access policy. Memory security can also be tested at compile time through tools like Valgrind to minimize the risk of memory-related vulnerabilities.

Crypto Hash:

The system leverages WolfSSL/Simple Crypto, which provides a suite of cryptographic algorithms available for implementation. In the absence of a Trusted Third Party (TTP) (Fig. 1), a PRF-based Key Derivation Tree rooted in global.secrets is used. Authentication is facilitated through an HMAC-SHA256 protocol that requires a cryptographic group key per HSM accompanied by a user PIN. For secure key exchange, the Elliptic Curve key exchange protocol is employed to establish shared secrets between communicating parties.

How CryptoHash is implemented on SoC:

Cryptographic operations are integrated into the SoC via a hardware-assisted derivation and verification sequence. SHA-256 is utilized for GGM tree expansion, Group ID verification, and also a session binding HMAC that includes the current nonce or counter. Access attempts involving invalid PINs or unauthorized Group IDs trigger a mandatory 5-second hardware timer delay to mitigate brute-force attacks. Data-at-rest protection is achieved by storing a "Golden Reference" HMAC digests in non-volatile storage, ensuring plaintext HSM_PIN secrets are never exposed in flash memory.

Protected File Access

In the File Allocation Table, use this structure:

- File UUID: 16 bytes

- Group ID: 16 bits
- Permissions: 1 byte
- Counter: 4 Bytes
- Metadata HMAC: 32 bytes, HMAC(GroupKey, UUID + GroupID + Permissions + Counter)

After the HSM verifies the HSM_PIN, it checks whether the user has access to the requested permissions. To calculate the metadata HMAC from the firmware side, you just need to calculate the GroupKey which can be done from the tree using the GroupID. Once the firmware calculates the metadata HMAC, it compares it in constant time to the metadata HMAC in the FAT. This is done to prove that the permissions haven't been tampered with. Finally to check what access the associated Group has to the file, it checks the Permissions byte.

System Integrity & Replay Protection

A 32-bit monotonic counter is stored in non-volatile flash memory to ensure it persists after the device restarts. Every time the HSM processes a write or receive command it increments the counter. When a new packet arrives, the HSM compares the packet's counter with the counter stored in flash, if it's not strictly greater than the counter, it's treated as a replay. The counter value is included in the metadata HMAC so an attacker cannot easily change one value in flash without breaking the signature. The firmware updates a Global Master Counter stored in flash storage and in the Per-File Counter in the FAT at the same time.

Nonces are generated using the hardware TRNG and stored in volatile SRAM and it's used to prove that the Host Tool is currently active. Before sending a command like Read, the host tool calls the Interrogate command and the HSM generates a 16 byte random nonce and sends it to the Host. The Host must include this nonce in the header of the command packet. If the packet's nonce doesn't match the value in SRAM, it isn't executed. Once a nonce is used or if a session times out, it is cleared to prevent reuse

Offset	Field	Size	Purpose
0x00	MAGIC	2 Bytes	Synchronization bytes (0xEC 0xTF)

0x02	OPCODE	1 Byte	Command ID (e.g., R for Read)
0x03	COUNTER	4 Bytes	Monotonic Counter (for Write/Receive)
0x07	NONCE	16 Bytes	Random Nonce (from Interrogate)
0x17	LEN	2 Bytes	Length of the encrypted payload
0x19	HMAC	32 Bytes	HMAC(GroupKey, Header + Payload)